



Mini projet data engineering FC Metz

NATHAN PLONTZ

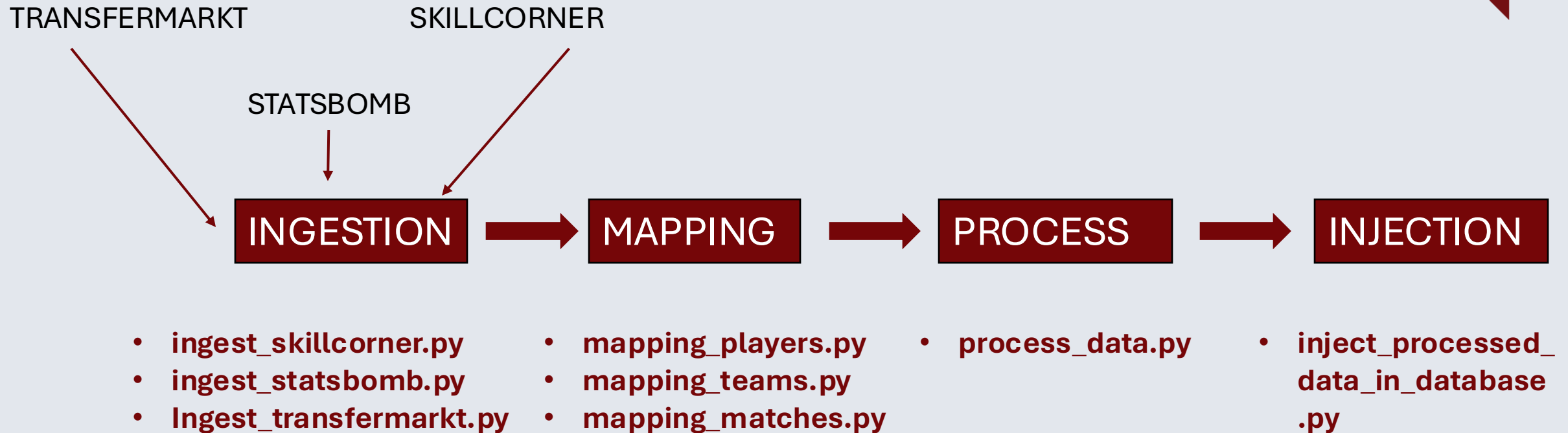


1. DESCRIPTION DU PIPELINE
2. INGESTION
 - TRANSFERMARKT
 - SKILLCORNER
 - STATSBOOMB
3. MAPPING
 - JOUEURS
 - ÉQUIPES
 - MATCHS
4. DATA PROCESS
5. BASE DE DONNÉES
6. DIFFICULTÉS RENCONTRÉES
7. AMÉLIORATIONS
8. CODE



1. DESCRIPTION DU PIPELINE

Pipeline.py



Un **exemple de lancement du pipeline** pour récupérer les **données matches de statsbomb**, récupérer les **données du joueur 36 de transfermarkt** et les **données teams de skillcorner** (voir l'ensemble des paramètres possibles dans le README sur git)

python pipeline --ingest statsbomb --matches --ingest transfermarkt --players 36 --ingest skillcorner --teams



2. INGESTION

Ingest_transfermarkt.py



Récupération des joueurs

- Id (TM)
- Name
- Link
- Age
- birth_date
- birth_place
- Nationalities
- height_cm
- current_club
- position
- shirt_number
- joined_date
- contract_end
- market_value_m
- Matches
- Goals
- Assists
- Minutes
- yellow_cards
- red_cards

Récupération des équipes

- Id
- team,link
- League
- Season
- squad_size
- average_age
- national_team_players
- stadium_name
- stadium_capacity
- table_position
- years_in_league

- **Récupération des équipes de ligue 1 puis récupération des joueurs actuelles des équipes**
- **Réalisé avec les packages request et BeautifulSoup**
- **Requêtes espacées d'un temps aléatoire entre 1s et 3s (éviter de voir les requêtes échouées)**
- **473 joueurs, 18 équipes**
- **Temps total : 28 min**

Ingest_skillcorner.py

Fc Metz



Récupération des joueurs

- Id (SC)
- first_name
- last_name
- short_name
- birthday
- trackable_object
- gender

Récupération des équipes

- Id (SC)
- name
- coach
- stadium_name
- stadium_city
- stadium_capacity

Récupération des matchs

- Id (SC)
- date_time
- home_team
- away_team
- status
- competition_id
- season_id
- competition_edition_id
- données spécifiques au match ...

Récupération des données physiques

- player_id
- match_id
- competition_edition_id
- minutes_full_all
- physical_check_passed
- total_distance_full_all
- total_metersperminute_full_all
- running_distance_full_all
- hsr_distance_full_all
- hsr_count_full_all
- sprint_distance_full_all
- sprint_count_full_all
- ...

- **Données physiques par joueur, pas par match, nommage "player_id_nbmatches_records.json"**
- **575 joueurs, 18 équipes, 207 matchs**
- **Temps total : 14 min**



Ingest_statsbomb.py

Récupération des joueurs

- Id (SB)
- team_id
- season_id
- competition_id
- birth_date
- Player_weight
- Player_height
- player_season_minutes
- player_season_npxg_per_shot
- ...

Récupération des équipes

- Id (SB)
- team_name
- season_id
- competition_id
- team_female
- team_season_matches
- team_season_minutes
- team_season_goals
- ...

Récupération des matchs

- Id (SB)
- Match_date
- Kick_off
- competition
- season
- Home_team
- Away_team
- Home_score
- Away_score
- Attendance
- Match_week
- behind_closed_doors
- ...

Récupération des données events

- Id(SB)
- player id
- match id
- period
- minute
- Seconde
- type
- Location_x
- Location_y
- duration
- possession_team
- under_pressure
- ...

- **Données events par match, pas par joueur, nommage "match_id_events.json"**
- **Récupération uniquement des matchs joués** (match_status == "available")
- **527 joueurs, 207 matchs, 18 équipes**
- **Temps total : 11 min**



3. MAPPING

mapping_teams.py

Exemple: Olympique Lyon(TM), Lyon(SB), Olympique Lyonnais (SC)

Solution



Algorithme hongrois :

- Construire une matrice 18×18 de scores de similarité
- Trouver l'assignation globalement optimale
- Garantit qu'un club ne peut être assigné qu'une seule fois

Score de similarité (package rapidfuzz) :

- combinaison pondérée de trois métriques
- Token_set : ignore les mots en double, gère les noms partiels
- Token_sort : réordonne les mots avant de comparer, gère les inversions
- Ratio simple : comparaison caractère par caractère, séquence brute

Matching :

- D'abord SC↔TM
- Puis SB↔(paires SC/TM)
- fusionne d'abord des deux sources les plus similaires en nomenclature

➤ **Après vérification, matching parfait**

mapping_players.py



Exemple:

- Marcos Aoás Corrêa(1994-05-14)(SB), Marquinhos(1994-05-14)(SC), Marquinhos(1994-05-14)(TM)
- Ali Yousef Ibrahim Al Musrati(2001-07-09)(SB), Ali Youssef Musrati(2001-07-09)(SC), Ali Youssif(2001-07-09)(TM)
- Sambou Soumano (2001-01-13)(SB), Sambou Soumano (2001-01-03)(SC), Sambou Soumano (2001-01-03)(TM)
- Djaoui Cissé (2004-01-30)(SB), D. Cissé (2004-01-31)(SC), Djaoui Cissé (2004-01-31)(TM)

Prétraitement : suppression des accents, passage en minuscules, suppression de la ponctuation

Algorithme

SI **nom + Date de naissance** correspondent exactement

→ MATCH (Exact)

SINON SI **similarité**(nom) ≥ 95

→ MATCH (Fuzzy nom)

SINON SI même **Date de naissance ET similarité**(nom) ≥ 45

→ MATCH (Fuzzy + DDN)

SINON SI même **Date de naissance ET similarité**(surnom / short_name) ≥ 95

→ MATCH (Alias + DDN)

SINON

→ NON MATCHÉ



Similarité fait avec rapidfuzz

- Mapping forcément imparfait : 473 joueurs(TM), 575 joueurs(SC), 527 joueurs(SB)
- Les sources possèdent des joueurs que d'autres ne possèdent pas et vice-versa (joueur transféré, joueur n'ayant pas encore joué)
- Certaines dates de naissance incorrectes
- Après vérification manuelle (et par rapport aux sources disponibles) mapping réussi excepté 1 joueur

mapping_matches.py



- Mapping matches entre SB et SC
- Comparaison de l'id des teams (à l'aide de mapping teams préalablement fait et exact) et de la date du match
- Simple grâce au mapping de teams
- SB et SC ne publient pas les matchs au même rythme → le mapping ne peut lier que les matchs **disponibles dans les deux sources**



4. DATA PROCESS

process_data.py



Du brut au propre

- Données brutes issues de 3 sources → formats hétérogènes (JSON, CSV, structures imbriquées)
- Fusion via les tables de mapping → **unification sous un ID propre au projet**
- Events StatsBomb aplatis et éclatés en sous-tables prêtes à injecter
- **Output : des CSVs propres, directement injectables dans la DB**
 - edition.csv
 - events.csv
 - events_ball_recovery.csv
 - events_carry.csv
 - events_pass.csv
 - events_pressure.csv
 - events_shot.csv
 - match_players.csv
 - matches.csv
 - physical.csv
 - players.csv
 - teams.csv



5. BASE DE DONNÉES



Pourquoi PostgreSQL ?

- Données interconnectées → base relationnelle naturellement adaptée
- Foreign keys → intégrité garantie par la base elle-même
- Robuste, open source, standard en production

Pourquoi la table match_players ?

- Relation classique en base de données "Many-to-many" : un joueur joue plusieurs matchs, un match a plusieurs joueurs
- Un joueur seul ou un match seul → pas de contexte, match_players les relie : *"ce joueur, dans ce match"*

Pourquoi des sous-tables events ?

- Socle commun (minute, joueur, position...) + champs spécifiques par type
- 1 table commune + 1 sous-table par type → **Pas de colonnes inutilement nulles**

Pourquoi events et physical se rattachent à match_players ?

- Chaque event et chaque donnée physique appartient à **un joueur précis** dans **un match précis**
- Parfaitement caractérisés par la relation match_players

<https://dbdiagram.io/d/699b8fd2bd82f5fce272c27a>

6. DIFFICULTÉS RECONTRÉES



DIFFICULTÉS RECONTRÉES

- **Peu de temps** — ~8 jours pour tout concevoir et implémenter
- **Anti-bot Transfermarkt** — risque de blocage des requêtes, délais aléatoires obligatoires
- **Temps de réponse** — appels API StatsBomb, SkillCorner et scraping Transfermarkt chronophages
- **Mapping joueurs imparfait** — nombre de joueurs différent entre les sources, couverture jamais à 100%
- **Dates de naissance erronées** — une source peut avoir une date de naissance incorrecte, ce qui casse le matching
- **Surnoms et noms composés** — chaque source a ses propres conventions → "*Marquinhos*" chez l'une, "*Marcos Aoás Corrêa*" chez l'autre
- **Volume des données events** — des centaines de milliers de lignes, une par action en jeu
- **Métriques difficiles à interpréter** — certains indicateurs nécessitent une expérience métier pour juger de leur pertinence
- **Recul sur la modélisation** — la structure de la DB évolue avec la connaissance des données, plus d'expérience nécessaire
- **Documentation dense** — trois sources de données à assimiler en parallèle en très peu de temps



7. AMÉLIORATIONS



AMÉLIORATIONS

- **Mapping** — avoir une source vérifiée qui fait le lien entre les IDs des différentes sources
- **Parallélisation des requêtes** — Multi-threading pour accélérer la récupération des données
- **Développer un front** — interface de visualisation connectée à la base (Streamlit)
- **Plus de temps sur les données** — mieux comprendre et exploiter certaines métriques métier
- **Ajout de tous les events** — actuellement seulement 5 types, étendre à l'ensemble des events StatsBomb
- **Extension à d'autres compétitions** — étendre le projet pas que a cette saison de ligue 1



8. CODE



CODE

```
C:\.
| .gitignore
| README.md
|
+---data
|   +---processed
|   |
|   \---raw
|       +---mapping
|       +---skillcorner
|       |   +---matches
|       |   +---physical
|       |   +---players
|       |   \---teams
|       +---statsbomb
|       |   +---events
|       |   +---matches
|       |   +---players
|       |   \---teams
|       \---transfermarkt
```

```
+---database
|   |   create_database.py
|   |   db_connection.py
|   |   inject_processed_data_in_database.py
|   |
|   +---scripts
|       ingest_skillcorner.py
|       ingest_statsbomb.py
|       ingest_transfermarkt.py
|       mapping_matches.py
|       mapping_players.py
|       mapping_teams.py
|       pipeline.py
|       process_data.py
```

<https://github.com/PLONTZNathan/Mini-projet-data-engineering---FC-Metz>